

Predictive Analysis and Proactive Intervention for Mitigating Cognitive Complexity Accumulation in Java Pull Requests

Submitted by:

IM/2021/028

Manditha Madushanka

Supervised by:

Prof. Janaka Wijayanayake

A dissertation submitted in partial fulfillment of the
Requirements for the Degree of Bachelor of Science in
Information Technology

Bachelor of Science (Hons) Degree in Information Technology

Department of Industrial Management

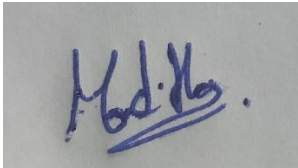
Faculty of Science

University of Kelaniya

Sri Lanka

Declaration

I hereby state that this research report and findings presented in it are my own and it has not been submitted before nor is it currently being submitted for any other academic program. Where material has been used from other sources due recognition has been given by mentioning the source.



.....

Signature of Candidate

Date: 17/07/2026

The above candidate has carried out research for the undergraduate dissertation under my supervision. The effective date of submission is confirmed below.

.....

Signature of Supervisor

Name: Senior Professor. Janaka Wijayanayake

Effective date of submission: 17/07/2026

Abstract

Cognitive complexity has become one of the dominant drivers of software maintenance cost, and software maintenance and evolution are estimated to consume between fifty and eighty per cent of the total cost of ownership of a typical enterprise system. In modern distributed development, the Pull Request has become the atomic unit of code evolution and the principal quality gate protecting the master branch, yet empirical evidence from mature open-source ecosystems shows that this gate leaks: a substantial majority of merged Pull Requests either introduce new technical debt or leave existing debt unchanged, and code smells that are direct manifestations of excess cognitive complexity are accepted into the master branch at rates comparable to clean contributions. The prevailing industrial response remains reactive, in that static-analysis tools report complexity only after code has been committed, by which time the contributor's cognitive context has already shifted and the cost of remediation is disproportionately high. Current research addresses the two halves of this problem in isolation: predictive models can flag problematic Pull Requests at submission time, and automated-refactoring techniques can generate concrete remedies, but no publicly reported framework fuses prediction with proactive intervention inside the Java Pull Request workflow itself.

In this research, the design, development, and evaluation of a Predictive and Proactive Code Health (PPCH) framework is presented, with the aim of mitigating the accumulation of cognitive complexity in Java projects by coupling predictive analysis of incoming Pull Requests with proactive intervention at submission time. The framework is organized around a Sense $\rightarrow$ Analyze $\rightarrow$ Act architecture. In the Sense layer, a multi-dimensional feature set spanning structural, process, and textual-semantic features is extracted from a Pull Request at the moment of submission, with cognitive complexity operationalized through the SonarSource Cognitive Complexity metric and supplemented by slice-based measures. In the Analyze layer, supervised machine-learning models, principally Random Forest and XGBoost, are trained to predict whether an incoming Pull Request will exceed the repository's cognitive-complexity threshold. In the Act layer, the prediction is translated into a proactive intervention delivered through a GitHub Action, either as a predictive complexity warning or as an

Integer-Linear-Programming-driven "Extract Method" refactoring suggestion, positioned deliberately at the Pull Request submission boundary where developer receptivity is highest.

Historical Pull Request data drawn from mature Apache Software Foundation Java projects are used for model development, with all features computed strictly from the state visible at submission time to prevent information leakage. The framework is evaluated through out-of-sample bootstrap validation of predictive performance, ablation studies isolating the contribution of each feature family, comparative benchmarking of Mean Time to Detect and Mean Time to Recover against a reactive SonarQube baseline, and a System-Usability-Scale-based assessment of developer receptivity. The principal contributions of this research are a paradigm of prediction-coupled proactive intervention for cognitive-complexity management and a lightweight, reproducible implementation of that paradigm inside the Java Pull Request workflow, shifting the center of gravity of quality assurance from reactive detection to proactive prevention. Future work would extend the framework to further object-level and class-level refactoring and to programming languages beyond Java.

Acknowledgements

I would like to thank Senior Professor. Janaka Wijayanayake who supervised this research, and provided the guidance, critical feedback and supportive encouragement that were instrumental in informing all major decisions and actions during the conduct of this research.

This thesis is based on the education received from the staff of the Department of Industrial Management, University of Kelaniya on software engineering, machine learning, and software quality, for which I am indebted. I extend my appreciation to all academic and non-academic members of the Department of Industrial Management, whose cooperation and assistance greatly contributed to the successful completion of this research.

Acknowledgement is also due to those responsible for the open-source tools and datasets used during this research in particular the Apache Software Foundation projects, the SonarSource Cognitive Complexity tooling, and the open-source metric

and refactoring implementations whose availability made this research reproducible for the software-engineering research community.

Acknowledgement is due to all the peer reviewers of the departmental research seminars, whose constructive comments during the interim presentations informed the research framing and evaluation.

Finally, I wish to thank my family and colleagues for their unwavering support, understanding, and encouragement, which have been vital in overcoming challenges and completing this work successfully.

Table of Contents

Declaration.....	i
Abstract.....	ii
Acknowledgements.....	iii
List of Tables	vii
List of Figures	viii
List of Acronyms	viii
Chapter 1 - Introduction.....	1
1.1 Chapter Introduction	1
1.2 Background and Motivation.....	1
1.3 Problem Context, History, and Real-World Impact.....	4
1.4 Research Objectives and Research Questions.....	6
1.5 Scope of the Research	7
1.6 Brief Summary of Anticipated Outcomes.....	8
1.7 Dissertation Structure Overview	9
1.8 Chapter Summary.....	9
Chapter 2 - Literature Review.....	10
2.1 Chapter Introduction	10
2.2 Review Methodology	10
2.2.1 Search Strategy and Data Sources	11
2.2.2 Inclusion and Exclusion Criteria	11
2.2.3 Study Selection and the PRISMA Flow	12
2.3 Software Complexity Metrics: From Structural to Cognitive.....	13
2.3.1 Structural Complexity and Its Limitations	13
2.3.2 Cognitively Grounded Complexity Metrics	14
2.3.3 Slice-Based Cognitive Complexity	15
2.3.4 Comparative Synthesis of Metrics.....	16
2.3.5 Relation to the Present Research	17
2.4 The Pull Request as the Locus of Complexity Accumulation	17

2.4.1 The Modern Role of the Pull Request	17
2.4.2 Technical Debt through the PR Channel	18
2.4.3 Predicting PR Outcomes.....	18
2.4.4 Relation to the Present Research	19
2.5 Machine Learning for Software Quality Prediction	19
2.5.1 The Predictive-Modelling Landscape.....	19
2.5.2 Supervised Classifiers for Defect and Complexity Prediction	19
2.5.3 Textual and Semantic Features.....	20
2.5.4 Comparative Synthesis	20
2.5.5 Relation to the Present Research	22
2.6 Automated and Semi-Automated Refactoring	22
2.6.1 Refactoring as Complexity Reduction.....	22
2.6.2 Broader Refactoring Tooling.....	23
2.6.3 Relation to the Present Research	23
2.7 Proactive AI Assistance and Developer Cognition	23
2.7.1 Cognitive Load Under AI Assistance.....	23
2.7.2 Developer Perception and Adoption	24
2.7.3 Relation to the Present Research	24
2.8 Research Gap Analysis and Synthesis.....	24
2.9 Conceptual Framework	27
2.10 Chapter Summary.....	28
Chapter 3 - Methodology	30
3.1 Chapter Introduction	30
3.2 Research Design and Philosophical Stance.....	30
3.2.1 Philosophical Stance.....	30
3.2.2 Research Design	30
3.3 Data Collection Approach.....	31
3.3.1 Target Population and Sampling	31
3.3.2 Data Sources and Extraction	31
3.3.3 Feature Engineering.....	32

3.3.4 Data Cleaning and Class-Imbalance Handling.....	32
3.4 Framework Development Process.....	33
3.4.1 Overall Architecture	33
3.4.2 Predictive Model Development.....	33
3.4.3 Automated Refactoring Engine	34
3.4.4 CI/CD Integration.....	34
3.5 Validation and Evaluation Approach.....	35
3.5.1 Quantitative Model Evaluation.....	35
3.5.2 Ablation Studies	35
3.5.3 Comparative Benchmarking.....	35
3.5.4 Usability Assessment.....	36
3.5.5 Threats to Validity	36
3.6 Ethical Considerations	36
3.7 Chapter Summary.....	37
References	38

List of Tables

Table 1: Comparative summary of software complexity metrics reviewed in this study	16
Table 2: Machine-learning approaches to software-quality prediction reviewed in this study.....	21
Table 3: Coverage of the five capability dimensions of the proposed framework by the reviewed literature	25

List of Figures

Figure 1: PRISMA flow diagram of the study-selection process across the four searched sources (ACM Digital Library, IEEE Xplore, Google Scholar, and ResearchGate)...	13
Figure 2: Conceptual framework of the proposed Predictive and Proactive Code Health (PPCH) framework	28

List of Acronyms

Acronym	Definition
ACM	Association for Computing Machinery
AI	Artificial Intelligence
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
ASF	Apache Software Foundation
AUC	Area Under the Receiver Operating Characteristic Curve
BCS	Basic Control Structures
BERT	Bidirectional Encoder Representations from Transformers
CI/CD	Continuous Integration and Continuous Deployment
DevOps	Development and Operations
GCN	Graph Convolutional Network
IEEE	Institute of Electrical and Electronics Engineers
ILP	Integer Linear Programming
LLM	Large Language Model
ML	Machine Learning
MNB	Multinomial Naïve Bayes
MTTD	Mean Time to Detect
MTTR	Mean Time to Recover
OO	Object-Oriented
PPCH	Predictive and Proactive Code Health
PR	Pull Request

PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
REST	Representational State Transfer
RF	Random Forest
ROC	Receiver Operating Characteristic
RQ	Research Question
SDG	Sustainable Development Goal
SMOTE	Synthetic Minority Over-sampling Technique
SSCC	SonarSource Cognitive Complexity
SUS	System Usability Scale
VAE	Variational Autoencoder
XGBoost	Extreme Gradient Boosting

Chapter 1 - Introduction

1.1 Chapter Introduction

This chapter establishes the intellectual and practical foundations upon which the present research is built. The background against which the research is situated is first outlined so that the reader is oriented to the technical, economic, and cognitive forces that motivate the investigation. The problem context is then framed against the historical evolution of software complexity metrics and the contemporary shift towards Continuous Integration and Continuous Deployment (CI/CD) practices in enterprise Java development. From this framing, the research objectives, research questions, and the scope of the study are derived. A concise preview of the principal findings is provided, and the overall structure of the dissertation is described so that the reader is able to navigate the document with a clear expectation of what each subsequent chapter contributes.

1.2 Background and Motivation

Modern software systems have grown to a scale, a level of distribution, and a rate of change that impose unprecedented cognitive demands on the individuals who write, review, and maintain them [Noor et al., 2025]. Complexity in software is not merely a structural property of the artefact but is now widely recognized as a psychological property of the human developer's interaction with it, since the human capacity to hold interacting concepts in working memory is bounded and finite [Ajeigbe and Emma, 2021]. When the mental effort required to comprehend a body of code exceeds those bounds, the phenomenon termed "information overload" is observed, and empirical evidence has repeatedly shown that this overload is a leading precursor to the introduction of defects, latent security vulnerabilities, and long-lived maintenance burdens [Alqadi and Maletic, 2020].

The economic weight of this cognitive dimension of software work is substantial. Software maintenance and evolution are estimated to consume between fifty and eighty per cent of the total cost of ownership of a typical enterprise software system, and cognitive complexity has been repeatedly identified as one of the dominant drivers of

that spend because of its strong empirical correlation with both comprehension time and defect proneness [Ajeigbe and Emma, 2021; Saborido et al., 2022]. The overhead of comprehending code is not an incidental cost but a structural one; every act of debugging, extending, or refactoring a module is preceded by an act of understanding, and each incremental unit of complexity in the artefact translates into additional developer minutes, elevated error rates, and a heightened likelihood that a subsequent modification will further degrade the artefact [Muñoz Barón et al., 2020; Calixto et al., 2025].

Traditional structural metrics, most prominently McCabe's Cyclomatic Complexity, quantify the intricacy of a program by counting the linearly independent execution paths through its control flow graph [Odeh et al., 2024]. Although such metrics have long served as a lingua franca between practitioners and static-analysis tools, they have been repeatedly critiqued for failing to reflect the actual mental effort demanded by the human reader of the code [Alqadi and Maletic, 2020; Gonzalez-Disla, 2023]. Cognitive informatics theory offers a more faithful account by treating source-code comprehension as a bottom-up process in which the developer incrementally builds mental "chunks" from statements, expressions, and control structures, subject to the well-known short-term memory capacity of roughly four such chunks at a time [Alqadi and Maletic, 2020; Auprasert and Limpiyakorn, 2011].

In response to this critique, more human-centered metrics have been proposed. Cognitive weights for basic control structures, in which nested loops and cascading conditionals are charged more heavily than sequential statements, provide a foundation for approximating the intellectual effort of program comprehension [Shehab et al., 2015; Gonzalez-Disla, 2023]. Building upon this foundation, the SonarSource Cognitive Complexity (SSCC) metric operationalizes three principles ignoring purely syntactic shorthand, incrementing for each interruption of linear control flow, and penalizing deeply nested structures relative to the enclosing method and has thereby become the de-facto industry metric for the "understandability gap" left uncovered by cyclomatic measures [Campbell, 2018; Muñoz Barón et al., 2020; Saborido et al., 2022]. A large empirical validation combining ten independent studies, comprising 427 code snippets and roughly 24,000 human evaluations, has established that cognitive complexity correlates significantly with comprehension time with a mean coefficient of 0.654, thereby providing the strongest evidence to date that any code-based metric

aligns with human perception [Muñoz Barón et al., 2020]. Complementing these method-level measures, slice-based cognitive complexity metrics operating on program slices have been shown to correlate significantly with defect probability, and a proportional increase in cognitive complexity has been associated with an increase in defect density in ninety-three per cent of open-source systems examined [Alqadi and Maletic, 2020; Maletic and Alqadi, 2020].

The industrial context in which these metrics are now applied has itself undergone a profound transformation. The Pull Request (PR) has emerged as the atomic unit of code evolution in modern distributed development, particularly on the GitHub platform, and it functions in practice as the principal quality gate protecting the master branch of a repository [Azeem et al., 2024]. Yet empirical evidence indicates that this gate leaks. Exploratory work on twenty-five widely used Java projects has documented that 32.3 per cent of examined PRs introduced "God Class" smells and 20 per cent introduced "Long Method" smells, both of which are direct manifestations of excess cognitive complexity [Azeem et al., 2024]. A separate study of Apache Software Foundation projects, covering 2,035 PRs, established that 75.56 per cent of merged PRs either introduced new technical debt or left the existing debt unchanged, a finding that undermines the naive assumption that peer review alone will discipline complexity accumulation [Calixto et al., 2025]. "Smelly" PRs are further associated with longer merge latencies and more protracted review discussions, and yet they are ultimately accepted at rates comparable to clean PRs, so that the cumulative effect on the master branch is one of steady degradation over time [Azeem et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023].

Against this backdrop, the maturing field of Machine Learning (ML) applied to software engineering has begun to reshape how quality is assured. A systematic survey of 421 papers published between 2009 and 2020 documented an accelerating adoption of deep learning models for classification problems in software maintenance, in particular for defect and quality-attribute prediction [Yang et al., 2022]. Supervised approaches, exemplified by Multinomial Naïve Bayes, have been reported to reach up to 97.3 per cent accuracy for the prediction of structural complexity levels [Odeh et al., 2024]. Multi-output deep-learning models have been developed to predict simultaneously the acceptance, review latency, and reopening probability of a PR at submission time, exhibiting substantive accuracy gains when tabular features are

combined with textual embeddings [Banyongrakkul and Phoomvuthisarn, 2023]. Recent advances in intelligent automation of software engineering activities have further demonstrated that automated pipelines can shift meaningful cognitive load away from human developers when properly integrated with the developer's workflow [Noor et al., 2025; Hassan et al., 2025].

Alongside these predictive advances, the mechanics of complexity reduction itself have matured. Integer Linear Programming (ILP) formulations of the "Extract Method" refactoring have demonstrated that overly complex Java methods can be decomposed automatically into cognitively manageable sub-methods while preserving semantics, and empirical evaluation has reported a developer acceptance rate of 77 per cent for the resulting refactorings [Saborido et al., 2022; Saborido et al., 2024]. Field studies of proactive AI assistants indicate that developers welcome such tools as a "second pair of eyes" and are markedly more receptive to interventions that arrive at natural workflow boundaries such as PR submission - than to interruptions in the middle of active coding [Al Haque et al., 2025; Sergeyuk et al., 2026].

Notwithstanding these advances, a critical rift persists between the two halves of the ecosystem. On one side sits a reactive quality-assurance stack SonarQube, PMD, Checkstyle and comparable tools that reports complexity issues only after the code has been committed or written and whose recommendations agreement across tools is known to be low [Dossas, 2025]. On the other side sits a growing body of predictive research that can, in principle, flag problematic PRs at submission time, together with a growing body of automated-refactoring research that can, in principle, generate concrete remedies. What is absent from the current state of the art is a production-ready framework that fuses these two capabilities so that the prediction of a cognitive-complexity spike is coupled to a proactive intervention delivered inside the PR workflow itself. It is this "Fusion Gap" between prediction and prevention that the present research addresses.

1.3 Problem Context, History, and Real-World Impact

Historically, the measurement and management of software complexity has evolved in three overlapping waves. The first wave, structural measurement, arose in the late 1970s and 1980s with metrics such as McCabe's Cyclomatic Complexity and Halstead's

software science, and it continues to dominate mainstream static-analysis tooling to this day [Odeh et al., 2024]. The second wave, cognitive measurement, emerged from cognitive informatics research and produced metrics grounded in the observable behaviour of programmers under comprehension tasks, culminating industrially in SonarSource Cognitive Complexity and academically in slice-based cognitive metrics [Alqadi and Maletic, 2020; Muñoz Barón et al., 2020; Campbell, 2018]. The third and current wave attempts to bring intelligent, learning-based methods to bear on the same underlying phenomena, transforming complexity management from a diagnostic to a predictive discipline [Yang et al., 2022; Noor et al., 2025].

Despite the maturity of the first two waves and the acceleration of the third, the practice of complexity management inside real engineering organizations remains stubbornly reactive. In the current industrial norm, the CI/CD pipeline is configured to run static-analysis tools after a commit has been pushed or a PR has been opened, and the developer receives complexity warnings once their cognitive context has already shifted to the next task [Saborido et al., 2024]. The result is a well-documented "prevention gap" in which even acknowledged complexity issues persist because the refactoring cost, borne by an already-loaded contributor, exceeds the perceived immediate benefit [Saborido et al., 2022; Azeem et al., 2024]. The empirical consequence of this gap is severe: 75.56 per cent of PRs in mature Apache projects were shown to either introduce or fail to reduce technical debt [Calixto et al., 2025], and 37 per cent of accepted PRs still carried at least one code smell into the master branch [Azeem et al., 2024].

The real-world impact of these dynamics is not confined to abstract "code health" metrics. Increased cognitive load has been correlated with a measurable decrease in the correctness of comprehension tasks and with observable physiological markers of developer strain, including changes captured by electroencephalography and eye-tracking studies of AI-assisted coding [Muñoz Barón et al., 2020; Al Haque et al., 2025]. Studies of early-career developers indicate that current complexity metrics do not fully align with the perceived understandability of code, so that junior engineers frequently confront artefacts whose complexity is under-signalled by conventional tooling [Esposito et al., 2024]. This cognitive burden translates into elongated PR-review latencies, protracted discussions on smelly PRs, and heightened attrition risk for integrators charged with policing the master branch [Azeem et al., 2024;

Banyongrakkul and Phoomvuthisarn, 2023]. Considered at the portfolio level, the accumulation of complexity debt inflates the fifty-to-eighty per cent share of lifecycle cost attributable to maintenance and slows the pace at which digital infrastructure can evolve to meet business needs [Ajeigbe and Emma, 2021; Berciu and Moldovan, 2023].

The convergence of these forces a well-understood metric of cognitive complexity, an industrial workflow (the PR) that naturally lends itself to intervention, a maturing ML toolchain capable of prediction at submission time, and a formally sound automated-refactoring engine capable of remediation has created a rare opportunity to close the Fusion Gap. The present research is motivated by that opportunity and by the observation that no publicly reported framework has yet fused predictive analytics with intelligent automation inside the Java PR workflow to deliver a genuinely proactive quality gate.

1.4 Research Objectives and Research Questions

Framed against the problem context set out above, the overarching aim of this research, as expressed in its title Predictive Analysis and Proactive Intervention for Mitigating Cognitive Complexity Accumulation in Java Pull Requests was defined as the architecting and evaluation of a Predictive and Proactive Code Health (PPCH) framework for Java-based projects that couples predictive analysis of incoming Pull Requests with proactive intervention at submission time, thereby mitigating the accumulation of cognitive complexity in the master branch and shifting the centre of gravity of quality assurance from reactive detection to proactive prevention.

Four specific objectives were derived from this aim. The first objective was to explore and derive, from historical Java Pull Request data drawn from mature open-source projects, a multi-dimensional feature set spanning structural metrics, process metrics, and textual semantics that correlates with the occurrence of cognitive-complexity spikes at submission time [Azeem et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. The second objective was to build and train supervised Machine Learning models capable of classifying, at the moment of PR creation, whether an incoming Java Pull Request would exceed the SSCC threshold established for the target repository [Odeh et al., 2024; Saborido et al., 2022]. The third objective was to design and implement a Proactive Intervention Engine, integrated with the CI/CD pipeline through

GitHub Actions, that would emit real-time complexity warnings and Integer-Linear-Programming-driven "Extract Method" micro-refactoring suggestions prior to human review [Saborido et al., 2024; Hassan et al., 2025]. The fourth objective was to assess the efficacy of the resulting framework by benchmarking it against conventional reactive tooling in terms of Mean Time to Detect (MTTD), Mean Time to Recover (MTTR), and a qualitative usability assessment grounded in developer feedback [Dossas, 2025; Sergeyuk et al., 2026].

Four research questions were formulated to structure the investigation, and each was placed in direct correspondence with one of the four objectives stated above so that the study's questions, objectives, and methodology would remain mutually consistent. The first question, expressed formally as RQ1, asked what the most reliable predictors of cognitive-complexity spikes in Java Pull Requests were, when the candidate predictor space was permitted to span code features and process features drawn from the historical PR record [Azeem et al., 2024; Calixto et al., 2025]. The second question, RQ2, asked what accuracy could be achieved by supervised Machine Learning in predicting the introduction of "smelly" or overly complex code at the PR submission stage, given the multi-dimensional feature representation derived in answer to RQ1 [Banyongrakkul and Phoomvuthisarn, 2023; Odeh et al., 2024]. The third question, RQ3, asked what impact proactive interventions delivered at the PR-submission stage exerted on the cumulative accumulation of cognitive-complexity debt in the master branch over time [Saborido et al., 2024; Calixto et al., 2025]. The fourth question, RQ4, asked how developers perceived the usability of the proactive interventions, in terms of cognitive-load reduction and integration into their existing workflow [Sergeyuk et al., 2026; Al Haque et al., 2025]. Taken together, RQ1 and RQ2 correspond to the first and second objectives and address the predictive component of the framework, whereas RQ3 and RQ4 correspond to the third and fourth objectives and address, respectively, the effectiveness and the human acceptability of the proactive intervention component.

1.5 Scope of the Research

To preserve academic rigour and analytical depth, the study was bounded along five explicit dimensions. First, the research was restricted to the Java programming language, chosen for its dominance in large-scale enterprise systems and for the maturity of its static-analysis ecosystem [Azeem et al., 2024; Crasso et al., 2016].

Second, cognitive complexity was operationalised principally through SonarSource Cognitive Complexity, supplemented where appropriate by slice-based metrics that capture how logic is distributed across the PR change set [Muñoz Barón et al., 2020; Alqadi and Maletic, 2020; Saborido et al., 2022]. Third, the framework was designed to operate inside the GitHub Pull Request workflow, with GitHub Actions serving as the CI/CD integration surface for real-time interception of incoming code changes [Azeem et al., 2024; Phaithoon et al., 2024]. Fourth, historical data for model training and validation were drawn from open-source projects hosted by the Apache Software Foundation, chosen for the quality and longitudinal completeness of their historical PR records [Calixto et al., 2025]. Finally, the class of proactive interventions considered was restricted to predictive complexity warnings and automated "Extract Method" micro-refactorings driven by Integer Linear Programming [Saborido et al., 2024].

1.6 Brief Summary of Anticipated Outcomes

Because the empirical phases of the study are carried out after this interim submission, the outcomes summarised here are stated as the results that the framework is expected to yield rather than as findings already obtained. It is anticipated that cognitive-complexity spikes in Java Pull Requests will prove predictable from features observable at the moment of submission. Structural change features – most prominently the number of files modified, the volume of lines added, and the maximum method-level nesting depth introduced – are expected to dominate the top of the feature-importance ranking, whereas process features such as contributor tenure and prior acceptance rate are expected to provide a secondary but consistent lift, and textual embeddings of PR titles and descriptions are expected to contribute a smaller but detectable improvement to model performance. Taken together, these three families of features are expected to support a supervised classifier whose Area Under the ROC Curve exceeds the 0.80 target established a priori, in agreement with performance ranges previously reported for related classification problems in software engineering [Odeh et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023].

The Proactive Intervention Engine, once embedded in the CI/CD pipeline as a GitHub Action and configured to emit predictive warnings together with ILP-driven "Extract Method" suggestions at PR submission time, is expected to reduce the volume of overly complex code merged into the master branch and to shorten the Mean Time to Recover

for complexity issues relative to a reactive SonarQube baseline. Developer feedback collected through the usability assessment is expected to indicate that interventions placed at the natural workflow boundary of PR submission are perceived as substantively less disruptive than in-editor interruptions, in line with prior findings on the receptivity of developers to proactive AI at workflow boundaries [Sergeyuk et al., 2026; Al Haque et al., 2025].

1.7 Dissertation Structure Overview

The remainder of this dissertation is organized into six further chapters. Chapter 2 presents a systematic and thematically organized review of the literature on cognitive complexity, PR-based software evolution, machine-learning-based quality prediction, automated refactoring, and proactive developer tooling, and it identifies the specific "Fusion Gap" that the present research is designed to close. Chapter 3 sets out the methodology adopted for the study, including its philosophical stance, the research design, the data-collection and feature-engineering procedures, the framework-development process, the evaluation protocol, and the ethical considerations that governed all data handling.

1.8 Chapter Summary

This chapter has argued that the industrial practice of software quality assurance remains reactive at precisely the point in the development lifecycle where proactive intervention would be most effective, namely the moment at which a Pull Request is first submitted. Empirical evidence from mature open-source ecosystems was drawn upon to characterize the scale of the resulting complexity debt, and the maturing convergence of cognitive-complexity metrics, machine learning, and automated refactoring was identified as an opportunity to close the persistent "Fusion Gap" between prediction and prevention. Against this background, the research aim, four specific objectives, four research questions each mapped to its corresponding objective, and five scope boundaries were stated, a preview of the anticipated outcomes was provided, and the structure of the remaining chapters was described. Chapter 2 now turns to the systematic literature review that establishes the theoretical and empirical foundations of the framework developed in the chapters that follow.

Chapter 2 - Literature Review

2.1 Chapter Introduction

The purpose of this chapter is to establish, in a systematic and thematically organised manner, the body of existing knowledge upon which the present research is built and against which its contribution can be evaluated. Section 2.2 sets out the review methodology, documenting the search strategy, the sources searched, the inclusion and exclusion criteria, and the study-selection process through which the reviewed corpus was assembled. Five themes are then treated in turn. Section 2.3 examines the evolution of software complexity metrics, tracing the movement from purely structural measures to cognitively grounded ones. Section 2.4 focuses on the Pull Request as the modern locus of code evolution and the empirical evidence for its role in complexity accumulation. Section 2.5 surveys machine-learning approaches to software-quality prediction, giving particular attention to methods applicable at PR submission time. Section 2.6 reviews automated and semi-automated refactoring, with an emphasis on cognitive-complexity reduction and Integer Linear Programming formulations. Section 2.7 analyses the emerging literature on proactive AI assistance and developer cognition. Section 2.8 synthesizes the reviewed work into an explicit statement of the research gap, Section 2.9 develops the conceptual framework of the study from that synthesis, and Section 2.10 provides a summary of the chapter.

For each theme, the state of the art is summarized, approaches across papers are compared, and gaps relevant to the present research are highlighted. Every reviewed paper is drawn from the "Final Included_1" collection assembled for the study, and each work is explicitly related to the framework being developed in this dissertation.

2.2 Review Methodology

To ensure that the review reported in this chapter was systematic, transparent, and reproducible rather than merely narrative, a review protocol informed by the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines was designed and followed. The protocol specified the sources to be searched, the search terms to be applied, the criteria governing the inclusion and exclusion of studies, and

the sequence of screening stages through which candidate studies passed before being admitted to the final corpus.

2.2.1 Search Strategy and Data Sources

Four complementary sources were searched in order to balance breadth of coverage against academic sources: the ACM Digital Library, IEEE Xplore, Google Scholar, and ResearchGate. The search strings were constructed by combining terms describing the complexity construct ("cognitive complexity", "code complexity", "software complexity") with terms describing the unit of analysis ("pull request", "code review", "GitHub pull request"), terms describing the computational approach ("predictive analysis", "prediction model", "machine learning", "software analytics"), and, where an implementation focus was required, terms describing the programming context ("Java", "software repositories", "GitHub repositories"). Several Boolean variants of these term groups were applied iteratively across the four sources; the principal composite query conjoined the complexity, unit-of-analysis, computational-approach, and programming-context groups, while narrower variants substituted refactoring and complexity-reduction terms ("refactoring", "complexity reduction") or defect- and maintainability-oriented terms ("software defect prediction", "code quality prediction", "code smell", "maintainability") to probe adjacent subtopics. The reference lists of the most relevant papers were additionally examined so that further candidates could be identified through backward snowballing.

2.2.2 Inclusion and Exclusion Criteria

Studies were admitted to the review if they were written in English, were available in full text, and made an empirical or methodological contribution to at least one of the five thematic areas defined for the review, namely the measurement of software complexity, the quality of code introduced through pull requests, machine-learning-based prediction of software quality, automated or semi-automated refactoring, and proactive artificial-intelligence assistance for developers. Studies were excluded if they addressed domains unrelated to software source-code quality, if they were opinion pieces or editorials lacking supporting evidence, if they duplicated a study already captured from another source, or if their full text could not be obtained. Seminal earlier works that established foundational complexity metrics were retained even where they

predated the principal search window, because their exclusion would have obscured the intellectual lineage of the cognitive-complexity construct.

2.2.3 Study Selection and the PRISMA Flow

The records retrieved from the four sources were first consolidated and de-duplicated before being taken forward to screening, as summarized in the PRISMA flow diagram presented in Figure 1. A total of 138 records were identified across the four sources, comprising 53 records from the ACM Digital Library, 56 from Google Scholar, 25 from IEEE Xplore, and 4 from ResearchGate. Twenty duplicate records were removed before screening, leaving 118 records that were screened at the level of title and abstract against the inclusion and exclusion criteria; of these, 46 were excluded as not relevant to the defined thematic areas. The 72 reports thereby retained were sought for retrieval, of which 18 could not be obtained in full text, leaving 54 reports that were assessed in full text for eligibility. At this final stage 16 reports were excluded, 2 because they lacked adequate bibliographic details and 14 because they did not fall within the narrowed scope of the review. The 38 studies satisfying all criteria at the full-text stage constituted the final corpus that is reviewed thematically in the remainder of this chapter and that corresponds to the "Included" collection assembled for the study.

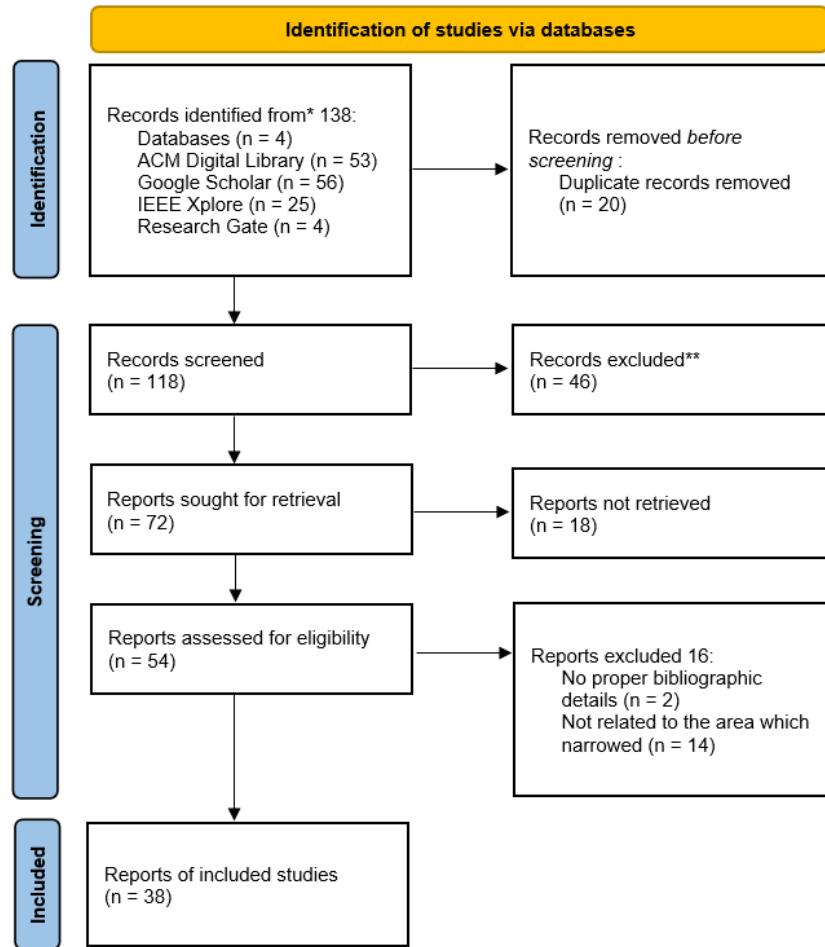


Figure 1: PRISMA flow diagram of the study-selection process

2.3 Software Complexity Metrics: From Structural to Cognitive

2.3.1 Structural Complexity and Its Limitations

The measurement of software complexity has historically been dominated by structural metrics, of which McCabe's Cyclomatic Complexity is the archetype. Cyclomatic Complexity quantifies the number of linearly independent paths through the control flow graph of a program unit and, by virtue of its computability from source or bytecode, has been embedded in essentially every mainstream static-analysis tool [Odeh et al., 2024]. Recent work has extended the tradition by using machine learning to predict Cyclomatic Complexity directly from source text, reaching high classification accuracy through the use of Multinomial Naïve Bayes and related

supervised methods [Odeh et al., 2024]. Analogous statistical treatments of code as a signal to be characterized by compound structural parameters have been proposed as generalizations of the structural tradition [Prokofiev and Savenko, 2023].

Despite the operational convenience of these structural measures, a recurring critique in the literature is that they capture the syntactic intricacy of the artefact but not the mental effort demanded of the human reader. It has been argued that a program with low Cyclomatic Complexity may nevertheless be extremely hard to understand if its control flow is deeply nested, its identifiers are opaque, or its data dependencies are widely distributed in the source [Alqadi and Maletic, 2020; Gonzalez-Disla, 2023]. This limitation motivated a shift towards metrics grounded in the cognitive science of program comprehension.

2.3.2 Cognitively Grounded Complexity Metrics

Cognitive complexity, in the technical sense used throughout this dissertation, was originally proposed as the mental effort demanded of a developer when performing tasks such as reading, testing, debugging, or modifying source code [Crasso et al., 2016]. Its theoretical foundation rests on cognitive informatics accounts of comprehension as a bottom-up chunking process bounded by the well-known short-term memory capacity of approximately four concurrent chunks [Auprasert and Limpiyakorn, 2011; Alqadi and Maletic, 2020]. Cognitive weights for the Basic Control Structures (BCS) framework, which assigns higher weights to nested constructs than to sequential ones, have been used to formalise this intuition and have been shown to align more closely with human intellectual effort than structural counts alone [Gonzalez-Disla, 2023; Shehab et al., 2015]. A unified perspective on complexity measurement, treating cognitive weights as first-class citizens alongside structural metrics, has been advanced as an integration of the two traditions [Jayakuru and Wijesinghe, 2023].

Two industrial-strength operationalisations of cognitive complexity dominate contemporary practice. SonarSource Cognitive Complexity (SSCC), developed by Campbell and adopted by SonarQube as its default cognitive-complexity metric, operationalizes three principles: purely syntactic shorthand is ignored, each interruption of linear control flow contributes an increment of one, and nested control structures incur a penalty proportional to their nesting depth relative to the enclosing method [Campbell, 2018; Muñoz Barón et al., 2020]. An accumulated cognitive complexity

metric has also been advanced, extending the principles of BCS weights to combined structural elements and offering an empirical benchmark against classical measures [Shehab et al., 2015]. Tool support for computing cognitive complexity in Java-based object-oriented systems has been demonstrated in industrial-scale hierarchies, showing that the metric can be collected reliably at scale [Crasso et al., 2016; De Silva et al., 2024].

The empirical validation of cognitive complexity as a predictor of human understandability has been the subject of a substantial meta-analysis combining data from ten independent studies covering 427 code snippets and roughly 24,000 human evaluations, which established that cognitive complexity significantly correlates with comprehension time at a mean coefficient of 0.654 and thereby offered the strongest evidence to date that a code-based metric aligns with human perception [Muñoz Barón et al., 2020]. Complementary work has explored perceptions of understandability among early-career developers, revealing that no single metric perfectly captures perceived complexity but that cognitive metrics outperform structural ones for the code artefacts most typical of production Java systems [Esposito et al., 2024].

2.3.3 Slice-Based Cognitive Complexity

A second strand of cognitively grounded measurement operates not on the whole method but on program slices subsets of statements identified by data and control dependencies with respect to a chosen variable. Slice-based metrics such as `sliceSize`, `sliceIdentifier`, and `sliceSpatial` quantify, respectively, the average tracing effort involved in comprehending a slice, the load imposed on short-term memory by programmer-defined identifiers appearing within it, and the lexical distance between definition and last use of a variable [Alqadi and Maletic, 2020]. These metrics have been shown to correlate significantly with defect probability across large corpora, and a proportional increase in slice-based cognitive complexity has been associated with an increase in defect density in ninety-three per cent of the open-source systems examined [Maletic and Alqadi, 2020; Alqadi and Maletic, 2020]. Program slicing has further been integrated with SSCC to yield hybrid defect-prediction models that combine method-level and slice-level views of cognitive load [Maletic and Alqadi, 2020].

2.3.4 Comparative Synthesis of Metrics

Table 1 summarizes the principal complexity metrics discussed in this section and situates them along the axes of structural versus cognitive grounding, tool availability, and empirical validation against human comprehension. As Table 1 shows, cognitive metrics and SSCC in particular, dominate the contemporary landscape both in tool availability and in the strength of the empirical validation that supports them.

Table 1: Comparative summary of software complexity metrics reviewed in this study

Metric family	Representative source	Grounding	Tool support	Empirical validation against human comprehension
Cyclomatic Complexity	[Odeh et al., 2024]	Structural (control flow)	Ubiquitous	Weak critiqued for missing cognitive effort [Alqadi and Maletic, 2020]
Basic Control Structures cognitive weights	[Gonzalez-Disla, 2023; Shehab et al., 2015]	Cognitive (control weighting)	Research-grade	Moderate - aligned with intellectual effort
Accumulated cognitive complexity	[Shehab et al., 2015]	Cognitive (combined)	Research-grade	Moderate
SonarSource Cognitive Complexity (SSCC)	[Campbell, 2018; Muñoz Barón et al., 2020]	Cognitive (understandability)	Industrial (SonarQube)	Strong mean correlation 0.654 with comprehension time [Muñoz

				Barón et al., 2020]
Slice-based cognitive metrics	[Alqadi and Maletic, 2020; Maletic and Alqadi, 2020]	Cognitive (data dependencies)	Research-grade (srcSlice)	Strong 93 % of systems show defect correlation
Object-oriented cognitive metrics	[Crasso et al., 2016; De Silva et al., 2024; Thamburaj and Aloysius, 2015]	Cognitive (OO extensions)	Java tooling	Moderate

2.3.5 Relation to the Present Research

The present research adopts SSCC as its primary metric on the strength of the meta-analytic evidence reviewed above and supplements it with slice-based metrics where the distributional properties of a PR change set demand a finer-grained view of how logic is spread across the modified code [Muñoz Barón et al., 2020; Alqadi and Maletic, 2020]. Unlike prior work, which has applied these metrics reactively after code has entered the master branch, the framework developed in this dissertation operationalizes them predictively at PR submission time so that they inform the intervention itself rather than merely diagnosing its aftermath [Saborido et al., 2022].

2.4 The Pull Request as the Locus of Complexity Accumulation

2.4.1 The Modern Role of the Pull Request

In distributed software development, the Pull Request has become the atomic unit of code evolution, functioning simultaneously as a review interface, a discussion thread, and a quality gate [Azeem et al., 2024]. Empirical work on twenty-five widely used Java projects has documented the frequency with which code smells enter the codebase

through the PR channel, reporting that 32.3 per cent of examined PRs introduced God-Class smells and 20 per cent introduced Long-Method smells [Azeem et al., 2024]. Studies of the readability of the code merged through this channel have shown that developers themselves prioritize naming and clarity in their improvement efforts and that only a fraction of the readability changes they make are captured by conventional tooling such as SonarQube [Dantas et al., 2023]. Related work on defect labelling has demonstrated that leveraging code-evolution information precisely the history exposed by the PR mechanism can substantially improve the quality of labels used in downstream code-smell datasets [Wang et al., 2024].

2.4.2 Technical Debt through the PR Channel

Technical debt accumulates through the PR channel at rates that peer review alone appears unable to discipline. A study of 2,035 PRs across Apache Software Foundation projects established that 75.56 per cent of merged PRs either introduced new technical debt or left the existing debt unchanged [Calixto et al., 2025]. This "prevention gap" is aggravated by the observation that smelly PRs, while attracting longer discussions and higher review latency, are ultimately accepted at rates comparable to clean PRs, so that the cumulative effect on the master branch is one of steady degradation over time [Azeem et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. Tool support for tracking self-admitted technical debt has been proposed at the PR level, with GitHub bots such as FixMe capable of detecting and monitoring on-hold self-admitted technical debt directly inside the PR workflow [Phaithoon et al., 2024]. Broader accounts of code smells as a determinant of software maintainability have reinforced the view that smells arising through this channel drive downstream maintenance cost [Lafi et al., 2020].

2.4.3 Predicting PR Outcomes

The predictability of PR outcomes has itself become a research theme. Multi-output deep-learning models have been used to predict the acceptance, review latency, and reopening probability of PRs simultaneously, exhibiting improvements when tabular process features are combined with textual embeddings of PR titles and descriptions [Banyongrakkul and Phoomvuthisarn, 2023]. Aggregate code-review analyses have similarly demonstrated the value of machine learning for classifying review comments and predicting reviewer disposition [Lal and Pahwa, 2022]. These lines of work establish that the PR event carries enough signal at submission time to support

supervised prediction, but they stop short of coupling those predictions to a proactive intervention capable of preventing the underlying defect.

2.4.4 Relation to the Present Research

The present research treats the Pull Request as its unit of analysis and intervention. Consistent with the empirical evidence that peer review alone leaves complexity debt largely unaddressed [Calixto et al., 2025; Azeem et al., 2024], the framework developed here does not seek to augment human review at the merge stage but rather to intervene before human review begins, exploiting the same features that prior work has shown to be predictive of PR outcome [Banyongrakkul and Phoomvuthisarn, 2023].

2.5 Machine Learning for Software Quality Prediction

2.5.1 The Predictive-Modelling Landscape

A systematic survey of 421 papers published between 2009 and 2020 documented the acceleration of machine-learning and deep-learning adoption in software-engineering prediction tasks, showing that deep-learning approaches convolutional and recurrent architectures in particular have progressively supplanted classical models for classification problems in software maintenance [Yang et al., 2022]. A broader account of intelligent automation in software engineering has argued that the field is now moving beyond isolated predictions towards integrated pipelines that combine prediction, refactoring, and developer feedback [Noor et al., 2025]. Complementary work has surveyed AI-driven defect prediction and automated refactoring as an integrated framework applicable to large-scale software systems [Hassan et al., 2025].

2.5.2 Supervised Classifiers for Defect and Complexity Prediction

At the level of individual classifiers, supervised methods such as Multinomial Naïve Bayes have been reported to achieve up to 97.3 per cent accuracy in predicting structural complexity levels [Odeh et al., 2024], while ensemble tree methods including Random Forest and gradient boosting have shown robust performance for defect and quality-attribute prediction across a wide range of software projects [Kapur and Sodhi, 2018; Singh and Bansal, 2023]. Object-oriented source-code metrics selected through a parallel genetic algorithm have been shown to improve predictive-model performance

when combined with careful feature selection [Vivanco and Jin, 2015]. Cognitive-complexity-aware defect-prediction models grounded in probabilistic reasoning have been proposed, establishing that increases in cognitive complexity translate into measurable increases in defect probability [Maletic and Alqadi, 2020]. Graph-convolutional models operating over the control-flow graph have been used to combine cognitive complexity with structural context for defect prediction, yielding improvements over both purely structural and purely cognitive baselines [Gupta et al., 2024].

2.5.3 Textual and Semantic Features

The feature space usable for predictive modelling extends beyond the tabular. Natural-language features drawn from PR titles, PR descriptions, commit messages, and inline comments have been exploited by multi-output deep-learning approaches, and the addition of pre-trained embeddings has been shown to lift predictive accuracy for PR-level outcomes [Banyongrakkul and Phoomvuthisarn, 2023]. Code-smell prediction using emerging Java language constructs has demonstrated that syntactic evolution of the language itself supplies useful signal to a supervised learner [Grodzicka et al., 2022]. Refactoring and maintainability prediction based on technical-debt issues has similarly benefited from the fusion of code and process features [Berciu and Moldovan, 2023].

2.5.4 Comparative Synthesis

Table 2 summarizes the reviewed machine-learning approaches to software-quality prediction along the axes of learning family, target attribute, feature modality, and reported performance regime. As Table 2 makes clear, existing work already demonstrates that predictive modelling of PR-level and method-level complexity is feasible; what it does not yet do is integrate the resulting predictions into a proactive intervention at PR submission time.

Table 2: Machine-learning approaches to software-quality prediction reviewed in this study

Learning family	Representative source	Target attribute	Feature modality	Reported performance regime
Naïve Bayes	[Odeh et al., 2024]	Structural complexity level	Source-text features	Up to 97.3 % accuracy
Random Forest / Gradient Boosting	[Kapur and Sodhi, 2018; Singh and Bansal, 2023]	Defectiveness	Tabular code + process	High accuracy, robust across projects
Genetic-algorithm feature selection + classifier	[Vivanco and Jin, 2015]	Defectiveness	OO source-code metrics	Improved predictive-model quality
Graph Convolutional Network	[Gupta et al., 2024]	Defectiveness	Control - flow graph + cognitive complexity	Improvement over tabular baselines
Deep learning (survey)	[Yang et al., 2022]	Multiple quality attributes	Code + text	Rising accuracy across 421 papers
Multi-output deep learning	[Banyongrakkul and Phoomvuthisarn, 2023]	PR acceptance / latency / reopening	Tabular + textual	Improved when modalities are combined

Logistic regression on slice metrics	[Alqadi and Maletic, 2020]	Defect probability	Slice-based cognitive metrics	Statistically significant across corpora
Emerging-construct-aware ML	[Grodzicka et al., 2022]	Code smells	Java syntactic features	Improved smell detection
Integrated AI defect-and-refactoring	[Hassan et al., 2025]	Defect + refactoring	Code, graph, LLM	Framework-level performance gains

2.5.5 Relation to the Present Research

The present research adopts supervised classifiers Random Forest and XGBoost in particular, trained on a multi-modal feature representation that combines structural, process, and textual features [Odeh et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. Class-imbalance mitigation through SMOTE and Variational Autoencoder augmentation is applied to address the natural minority status of smelly PRs [Banyongrakkul and Phoomvuthisarn, 2023]. Unlike the surveyed literature, however, the resulting models are not treated as endpoints but as the sensing layer of a proactive intervention, closing the loop between prediction and prevention.

2.6 Automated and Semi-Automated Refactoring

2.6.1 Refactoring as Complexity Reduction

Refactoring has long been recognised as the principal instrument for reducing cognitive complexity once it has accumulated. Recent work has formalised the automation of cognitive-complexity reduction as an optimisation problem, showing that Extract-Method refactorings can be systematically applied to overly complex Java methods to bring their SSCC below the recommended threshold [Saborido et al., 2022]. This line of work has been extended into a full Integer Linear Programming (ILP) formulation, in which the choice of statements to extract into new methods is cast as a constrained optimisation whose objective is to minimise SSCC while preserving semantics and

respecting nesting and data-dependence constraints [Saborido et al., 2024]. Reported evaluations of the ILP approach have demonstrated a developer-acceptance rate of 77 per cent for the resulting refactorings, indicating that automated refactoring is not merely feasible but also usable [Saborido et al., 2022].

2.6.2 Broader Refactoring Tooling

Tool support for cognitive-complexity analysis and optimisation in Java has been developed as a stand-alone research vehicle [De Silva et al., 2024]. AI-driven defect-prediction and automated-refactoring frameworks have been proposed that combine large language models with graph-based analyses to serve as "transformation agents" for autonomous refactoring [Hassan et al., 2025]. Comparative studies of code-quality tools have shown that different reactive tools excel at different smell categories, that inter-tool agreement is often low, and that even the most sophisticated static analysers require careful integration to be useful in modern DevOps environments [Dossas, 2025]. Visualisation-based approaches to complexity and maintainability, powered by AI-driven feature extraction, have further argued that developers require proactive maintenance strategies rather than post-hoc reports [Ajeigbe and Emma, 2021].

2.6.3 Relation to the Present Research

The present research adopts the ILP formulation of the Extract-Method refactoring as the remediation engine attached to its predictive component, reusing the semantics-preserving guarantees and optimization objective established in prior work [Saborido et al., 2022; Saborido et al., 2024]. Unlike prior work, the refactoring engine is not invoked as a standalone tool but is triggered conditionally by the classifier's prediction at PR submission time, so that its recommendations reach the developer at the moment of maximum influence rather than after human review has already begun.

2.7 Proactive AI Assistance and Developer Cognition

2.7.1 Cognitive Load Under AI Assistance

Empirical work on developer cognition in the age of AI assistants has begun to characterize the mental load imposed by AI-augmented workflows using physiological signals such as electroencephalography and eye-tracking, and has argued for the standardization of cognitive-load measurement in this new context [Al Haque et al.,

2025]. Field studies of proactive AI coding assistants over multi-day periods have found that developers welcome such tools as a "second pair of eyes" and that receptivity depends critically on the timing of the intervention, with interventions delivered at natural workflow boundaries such as PR submission significantly less disruptive than those delivered mid-implementation [Sergeyuk et al., 2026]. Lessons drawn from the engineering of the StackSpot AI contextualised coding assistant have further shown that response quality correlates strongly with the quality of the context supplied to the underlying model, arguing for retrieval-augmented approaches grounded in project-specific context [Pinto et al., 2024].

2.7.2 Developer Perception and Adoption

The perception of cognitive complexity by early-career developers has been investigated empirically, revealing that reliance on any single metric to convey code understandability is insufficient and motivating combined presentations that expose multiple complexity dimensions [Esposito et al., 2024]. Comparative studies of code-quality tooling and AI-generated code have emphasized that developer trust depends on tool transparency and on the plausibility of suggested remediations [Dossas, 2025]. Considered together, these studies imply that the effectiveness of a proactive intervention cannot be reduced to the accuracy of its underlying model but depends jointly on the timing, framing, and context of the recommendation.

2.7.3 Relation to the Present Research

The Proactive Intervention Engine designed in this dissertation is positioned deliberately at the PR-submission workflow boundary in accordance with the field-study evidence on developer receptivity [Sergeyuk et al., 2026] and integrates textual context from the PR title and description as advocated by contextualised-assistant work [Pinto et al., 2024]. Cognitive-load considerations informed the framing of the warnings and the presentation of the ILP-driven refactoring suggestions [Al Haque et al., 2025].

2.8 Research Gap Analysis and Synthesis

Across the five themes reviewed, a consistent pattern is discernible. Cognitively grounded complexity metrics have matured to industrial strength [Campbell, 2018; Muñoz Barón et al., 2020]. Pull Requests have become the operational unit at which complexity is introduced [Azeem et al., 2024; Calixto et al., 2025]. Machine-learning

models exist that can predict complexity and defect-related outcomes at PR submission time with usable accuracy [Odeh et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. Automated refactoring has been formalised as an ILP problem with a demonstrated developer-acceptance rate [Saborido et al., 2022; Saborido et al., 2024]. Proactive AI assistants have been shown to be welcomed by developers when they arrive at natural workflow boundaries [Sergeyuk et al., 2026; Al Haque et al., 2025]. Yet no single reviewed work fuses these five elements into an integrated, production-ready framework operating inside the Java PR workflow. This is the "Fusion Gap" that motivates the present research.

Table 3 maps the reviewed works against the five capability dimensions of the framework proposed in this dissertation cognitive metric grounding, PR-level applicability, predictive modelling, automated refactoring, and proactive workflow integration and demonstrates that while each dimension is individually well supported, no single prior work covers all five.

Table 3: Coverage of the five capability dimensions of the proposed framework by the reviewed literature

Representative source	Cognitive metric grounding	PR-level applicability	Predictive modelling	Automated refactoring	Proactive workflow integration
[Muñoz Barón et al., 2020]	Yes	No	No	No	No
[Alqadi and Maletic, 2020]	Yes	No	Partial	No	No
[Campbell, 2018]	Yes	No	No	No	No
[Azeem et al., 2024]	Partial	Yes	Partial	No	No

[Calixto et al., 2025]	Partial	Yes	No	No	No
[Banyongrakkul and Phoomvuthisarn, 2023]	No	Yes	Yes	No	Partial
[Odeh et al., 2024]	Partial	No	Yes	No	No
[Saborido et al., 2022]	Yes	No	No	Yes	No
[Saborido et al., 2024]	Yes	No	No	Yes	Partial
[Hassan et al., 2025]	Partial	Partial	Yes	Yes	Partial
[Sergeyuk et al., 2026]	No	Partial	No	No	Yes
[Al Haque et al., 2025]	No	No	No	No	Yes
Proposed PPCH Framework (this research)	Yes	Yes	Yes	Yes	Yes

Three specific gaps are exposed by this synthesis. First, the "Fusion Gap" already noted, in which prediction and prevention have not been coupled at PR submission time. Second, a "Metric-to-Workflow Gap", in which cognitively grounded metrics have been validated against human comprehension but have not been used as the objective of a proactive intervention embedded in the CI/CD pipeline. Third, a "Context Gap", in which textual semantics of the PR narrative are known to lift predictive accuracy but

have not been combined with structural and process features inside a framework that also intervenes in response.

2.9 Conceptual Framework

The conceptual framework that guides this research was derived directly from the gaps identified in the preceding synthesis and organizes the constructs drawn from the reviewed literature into a coherent, three-layered structure termed the Sense → Analyze → Act architecture. The framework is presented graphically in Figure 2 and is described in the text below so that the relationships among its constructs are made explicit.

The Sense layer corresponds to the observation of a Pull Request at the moment of its submission and to the extraction of the multi-dimensional feature set that the metrics and prediction literature has shown to carry signal about impending complexity. Three families of independent constructs populate this layer: structural features describing the size and control-flow characteristics of the change, process features describing the contributor's history and standing, and textual-semantic features describing the intent expressed in the PR title and description [Azeem et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. The complexity construct that these features are intended to predict is operationalised through SonarSource Cognitive Complexity, supplemented by slice-based metrics, in accordance with the validation evidence reviewed in Section 2.3 [Muñoz Barón et al., 2020; Alqadi and Maletic, 2020].

The Analyze layer maps the sensed features onto a predicted probability that the incoming PR will exceed the cognitive-complexity threshold of the target repository. This layer embodies the supervised-learning constructs reviewed in Section 2.5, in which structural, process, and textual features are fused within classifiers such as Random Forest and XGBoost [Odeh et al., 2024; Kapur and Sodhi, 2018]. The dependent construct produced by this layer is the complexity-spike probability, together with an explanation identifying the features that contributed most to the prediction.

The Act layer transforms the prediction into a proactive intervention delivered inside the developer's workflow. Two intervention constructs are defined: a predictive complexity warning and an automated "Extract Method" refactoring suggestion generated by the Integer-Linear-Programming engine reviewed in Section 2.6

[Saborido et al., 2022; Saborido et al., 2024]. The timing of the intervention at the PR-submission workflow boundary is itself a construct of the framework, grounded in the evidence that developer receptivity is highest at such boundaries [Sergeyuk et al., 2026]. The outcome constructs against which the framework is ultimately evaluated the volume of complex code merged into the master branch, the Mean Time to Detect and Mean Time to Recover, and the developer's perceived cognitive load close the framework by linking the intervention back to the code-health and human-centred objectives that motivated it [Calixto et al., 2025; Al Haque et al., 2025].

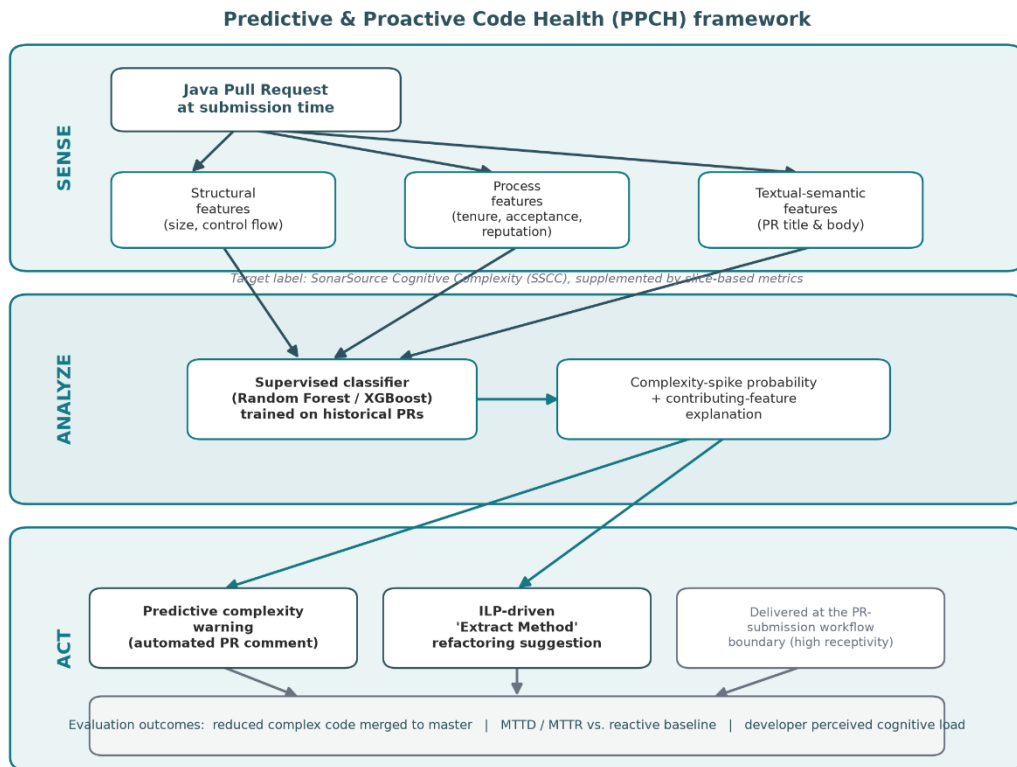


Figure 2: Conceptual framework of the proposed Predictive and Proactive Code Health (PPCH) framework

2.10 Chapter Summary

Following a systematic review protocol based on the PRISMA guidelines and the four-source search strategy documented at the outset, this chapter has reviewed the state of the art across the five themes most directly relevant to the present research: complexity measurement, the PR as a locus of code evolution, machine-learning-based quality prediction, automated refactoring, and proactive AI assistance. In each theme, the

reviewed works were compared and situated, and a gap was identified. Together, these gaps constitute the "Fusion Gap" between prediction and prevention that the present research is designed to close, and the constructs drawn from the reviewed literature were consolidated into the Sense → Analyze → Act conceptual framework presented in Figure 2. Chapter 3 now turns to the methodology adopted for that closure.

Chapter 3 - Methodology

3.1 Chapter Introduction

This chapter describes the research methodology that was designed and executed in order to investigate the research questions stated in Chapter 1 and to construct the framework whose motivation was established in Chapter 2. The philosophical stance of the research is first articulated, and the mixed-methods design that followed from it is described. The population, sampling strategy, and data-collection procedure are then presented, followed by the feature-engineering, model-development, and framework-construction processes. The evaluation strategy is set out next, and the ethical considerations that governed all handling of data and human participants are stated. The chapter closes with a summary. All work reported in this chapter is described in the past tense, consistent with its role as an account of research that has already been performed.

3.2 Research Design and Philosophical Stance

3.2.1 Philosophical Stance

The philosophical stance adopted for this research was that of pragmatism, informed by post-positivist commitments where quantitative claims were involved. Pragmatism was chosen because the research questions themselves demanded both quantitative measurement of predictive accuracy, of cognitive-complexity reduction, and of time-to-recover and qualitative interpretation of developer receptivity and cognitive load, and because the object of study, the Pull Request, is at once a technical artefact and a social event mediated by human review [Sergeyuk et al., 2026; Al Haque et al., 2025]. Post-positivist commitments were adopted for the quantitative components in the sense that hypotheses about predictive performance and framework impact were formulated a priori, evaluated against empirical evidence, and treated as falsifiable [Yang et al., 2022; Odeh et al., 2024].

3.2.2 Research Design

A sequential explanatory mixed-methods design was adopted. The design was structured as a Sense $\rightarrow$ Analyze $\rightarrow$ Act architecture, in which the "Sense" phase

corresponded to the collection and preparation of historical PR data, the "Analyze" phase to the training and validation of the predictive models, and the "Act" phase to the design and evaluation of the Proactive Intervention Engine embedded in the CI/CD pipeline. This design was chosen because it aligned naturally with the three-part structure of the framework being developed and because it permitted the quantitative results of the earlier phases to inform the qualitative usability evaluation conducted in the final phase [Noor et al., 2025; Hassan et al., 2025].

3.3 Data Collection Approach

3.3.1 Target Population and Sampling

The population of interest was defined as historical Pull Requests submitted to mature, actively maintained, open-source Java projects. To operationalize this population, the corpus was drawn from Apache Software Foundation (ASF) repositories, which were selected on the grounds of the completeness of their historical PR records, the maturity of their review culture, and their prior use in comparable empirical studies of technical debt and code smells [Calixto et al., 2025; Azeem et al., 2024]. Within this pool, a purposive sample of Java projects was drawn to ensure diversity across application domain (data-processing frameworks, web frameworks, and infrastructure libraries), project age, and contributor community size. All PRs opened during the sampled projects' history were included in the frame, and no PR was excluded on the grounds of its outcome, so that both accepted and rejected PRs contributed to the training and evaluation data [Azeem et al., 2024]. The sampling frame was scoped to yield a corpus of sufficient size to support supervised learning together with the class-imbalance handling described below, with the precise set of projects and the final PR counts fixed during the data-acquisition phase and reported in full in the chapter documenting the framework's construction.

3.3.2 Data Sources and Extraction

Historical PR data were extracted through the GitHub REST and GraphQL Application Programming Interfaces. For each sampled PR, the metadata retrieved included the title, the body, the change-set diff, the list of modified files, the timestamps of submission and merge (or closure), the identity and history of the contributor, and the full review discussion. Commit-level data were retrieved in parallel so that the exact

state of the codebase at PR submission time could be reconstructed and analyzed. SonarScanner was executed on the pre- and post-PR states of the codebase to extract SonarSource Cognitive Complexity values at method and class granularity and to identify code smells such as God Class and Long Method [Campbell, 2018; Saborido et al., 2022]. Slice-based cognitive metrics were computed on the same snapshots using the srcSlice tooling family [Alqadi and Maletic, 2020].

To prevent information leakage between the training and evaluation of the models, all features were computed strictly from the state of the world visible at the moment of PR submission. Post-submission signals such as reviewer comments, subsequent commits, or merge outcomes were used only as labels or held-out evaluation targets and never as inputs to the predictive model [Banyongrakkul and Phoomvuthisarn, 2023].

3.3.3 Feature Engineering

The multi-dimensional feature representation was constructed from three families. Structural features included the number of lines added and deleted, the number of files modified, the maximum nesting depth introduced by the change, the count of new control-flow constructs, and the delta in SSCC per modified method. Process features included the contributor's prior acceptance rate on the target project, the tenure of the contributor, the number of prior PRs submitted, and the reputation signal derived from the contributor's public follower count [Azeem et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023]. Textual-semantic features were derived from the PR title and body through pre-trained language-model embeddings, specifically BERT contextual embeddings and Word2Vec word embeddings, following the practice established in prior multi-output PR-outcome modelling and in contextualised coding-assistant work [Banyongrakkul and Phoomvuthisarn, 2023; Pinto et al., 2024]. Emerging-Java-construct signals were incorporated where relevant to preserve the syntactic information exploited by prior code-smell prediction work [Grodzicka et al., 2022].

3.3.4 Data Cleaning and Class-Imbalance Handling

Skewed distributions in the raw feature space were addressed through logarithmic transformation of heavy-tailed numeric features, following the practice adopted in comparable predictive-quality studies [Kapur and Sodhi, 2018]. The class imbalance inherent in the problem smelly, high-complexity PRs being a minority within the total

PR population was mitigated through the combination of Synthetic Minority Over-sampling Technique (SMOTE) and Variational Autoencoder-based augmentation on the training partitions, in line with the strategy adopted in prior multi-output PR modelling [Banyongrakkul and Phoomvuthisarn, 2023]. Feature selection was informed by prior evidence that carefully selected object-oriented metric subsets improve predictive-model performance [Vivanco and Jin, 2015].

3.4 Framework Development Process

3.4.1 Overall Architecture

The framework was designed around the Sense $\rightarrow$ Analyze $\rightarrow$ Act architecture. In the Sense layer, the GitHub Action listened for the pull request opened event, retrieved the change set and associated metadata, and extracted the multi-dimensional feature vector described above. In the Analyze layer, the feature vector was consumed by the supervised classifier trained on the historical corpus, and a probability that the PR would exceed the SSCC threshold was returned together with the top contributing features. In the Act layer, the probability and its explanation were translated into a proactive intervention, either a predictive complexity warning posted as an automated PR comment, or an automated "Extract Method" refactoring suggestion generated by the ILP engine [Saborido et al., 2022; Saborido et al., 2024].

3.4.2 Predictive Model Development

Candidate supervised classifiers were selected on the basis of their reported strength in comparable software-quality prediction tasks. Random Forest was included on the strength of its robust performance on tabular software metrics [Kapur and Sodhi, 2018; Singh and Bansal, 2023], XGBoost on the strength of its dominance in structured-data competitions and its demonstrated ability to handle heterogeneous features, and Multinomial Naïve Bayes on the strength of its reported 97.3 per cent accuracy for structural-complexity prediction [Odeh et al., 2024]. A graph-convolutional baseline was constructed for comparison in accordance with prior work integrating control-flow-graph structure with cognitive complexity [Gupta et al., 2024], and convolutional and recurrent deep-learning baselines were retained for the modelling of sequential PR patterns, consistent with the deep-learning trend documented across the surveyed literature [Yang et al., 2022]. Each model was trained on the augmented training

partition, tuned by grid search over its hyperparameter space, and evaluated on the held-out partition. Slice-based logistic-regression baselines were retained to preserve comparability with the defect-prediction literature [Alqadi and Maletic, 2020; Maletic and Alqadi, 2020].

3.4.3 Automated Refactoring Engine

The Extract-Method refactoring engine was implemented as an Integer Linear Program whose formulation was adapted from the semantics-preserving decomposition scheme established in prior automated cognitive-complexity-reduction work [Saborido et al., 2024]. Constraints encoded the preservation of data dependencies, the respect of nesting boundaries, and the requirement that no extracted fragment could depend on more than a fixed threshold of local variables. The objective function minimized the SSCC of the resulting decomposition subject to these constraints. Refactoring proposals were generated only when the classifier's predicted probability exceeded a configurable threshold, so that the ILP solver was not invoked on PRs whose predicted complexity was already acceptable [Saborido et al., 2022].

3.4.4 CI/CD Integration

The framework was packaged as a GitHub Action that could be installed on any Java repository by adding a workflow file to the repository's github/workflows directory. The Action was triggered on pull request events and, upon receiving a submission, computed the feature vector, ran the predictive model, and depending on the predicted probability posted a complexity warning, a set of "Extract Method" refactoring suggestions, or, when the prediction was benign, no intervention at all. The design of the intervention timing was informed by the field-study evidence that developers are markedly more receptive by approximately fifty-two per cent to proactive AI at natural workflow boundaries such as PR submission than during active editing [Sergeyuk et al., 2026]. The Action was engineered to be idempotent and to update its previous comment rather than to accumulate a new comment per subsequent commit to the PR, in line with GitHub-bot design practices established for technical-debt monitoring [Phaithoon et al., 2024].

3.5 Validation and Evaluation Approach

3.5.1 Quantitative Model Evaluation

In direct response to RQ2, the predictive performance of each candidate classifier was estimated using an out-of-sample bootstrap validation procedure repeated one thousand times, an approach chosen for its recognized superiority in yielding reliable performance estimates without overlap between training and testing partitions [Yang et al., 2022]. The primary quantitative metric was the Area Under the Receiver Operating Characteristic Curve (AUC), supplemented by precision, recall, F1 score, and calibration diagnostics. An a-priori target AUC of 0.80 and an accuracy band of 75 to 85 per cent were adopted, informed by the performance regimes reported in comparable software-quality classification work [Odeh et al., 2024; Banyongrakkul and Phoomvuthisarn, 2023].

3.5.2 Ablation Studies

Ablation studies were designed and executed to answer RQ1 by isolating the contribution of each feature family to the overall predictive performance and thereby identifying the most reliable predictors of complexity spikes. In each ablation configuration, one family structural, process, or textual-semantic was withheld from the feature vector, the model was retrained, and the resulting drop in AUC was recorded. This ablation protocol permitted a direct statement about which family of features contributed most to the prediction of cognitive-complexity spikes [Banyongrakkul and Phoomvuthisarn, 2023; Vivanco and Jin, 2015].

3.5.3 Comparative Benchmarking

To answer RQ3 and evaluate the framework as a whole against reactive baselines, a comparative benchmark was designed in which the same corpus of PRs was processed by the framework and by a SonarQube-based reactive pipeline. Mean Time to Detect (MTTD) and Mean Time to Recover (MTTR) of complexity issues were computed for each pipeline, together with the proportion of complexity-affected PRs merged into the master branch under each regime. The benchmark design followed established practice in comparative code-quality tool evaluation [Dossas, 2025]. The volume-of-complex-code-merged metric was computed against the master-branch history in accordance

with the definition of complexity accumulation as an incremental process operating on the master branch [Calixto et al., 2025].

3.5.4 Usability Assessment

To answer RQ4 and evaluate the framework's impact on developer cognition and workflow, a usability assessment was designed. Developer participants were invited to use the framework on realistic PR scenarios drawn from the sampled projects, and their perception of cognitive load, tool usefulness, and disruption was captured through a structured questionnaire modelled on the System Usability Scale. The design of the assessment was informed by prior field-study work on proactive AI assistants and by the standardization efforts around cognitive-load measurement in AI-assisted coding [Sergeyuk et al., 2026; Al Haque et al., 2025]. Complementary developer-perception evidence on complexity metrics was drawn upon in the framing of the questionnaire items [Esposito et al., 2024].

3.5.5 Threats to Validity

Threats to validity were considered explicitly. Threats to construct validity were addressed by grounding the operational definition of cognitive complexity in the meta-analytically validated SSCC metric [Muñoz Barón et al., 2020] and by triangulating it with slice-based metrics [Alqadi and Maletic, 2020]. Threats to internal validity were mitigated through the strict enforcement of pre-submission-only features and through the bootstrap validation protocol [Yang et al., 2022]. Threats to external validity, the risk that findings on ASF Java projects might not generalize to other ecosystems were acknowledged and framed as a scope boundary rather than a defect [Calixto et al., 2025; Azeem et al., 2024]. Threats to conclusion validity arising from label noise in the underlying dataset were mitigated by adopting the code-evolution-based labelling improvements advocated by prior work [Wang et al., 2024].

3.6 Ethical Considerations

Because the primary data source consisted of publicly available Pull Requests hosted on GitHub under permissive open-source licenses, no personally identifying information beyond that already published by contributors was collected, and no attempt was made to re-identify pseudonymous contributors. The purposive sampling frame was documented in full so that the study could be replicated. For the usability

assessment, informed consent was obtained from every participant prior to their engagement with the framework, and all responses were anonymized at the point of collection. The framework itself was designed so that its interventions would be transparent to the developer and the underlying probability, the top contributing features, and the rationale for any suggested refactoring were exposed alongside the intervention consistent with prior arguments that developer trust in AI-assisted tools depends on tool transparency [Dossas, 2025; Pinto et al., 2024]. Beyond human-subjects concerns, the research aligned itself explicitly with the United Nations Sustainable Development Goals, in particular Goal 8 (decent work through reduction of developer cognitive toil), Goal 9 (industry innovation through AIOps-oriented infrastructure), and Goal 12 (responsible consumption through the treatment of complexity debt as digital waste) [Noor et al., 2025; Ajeigbe and Emma, 2021].

3.7 Chapter Summary

This chapter has set out the research methodology that was designed and executed to investigate the research questions stated in Chapter 1 and to construct the framework whose motivation was established in Chapter 2. A pragmatist philosophical stance and a sequential explanatory mixed-methods design were adopted, and a Sense → Analyze → Act architecture was described. Purposive sampling of Apache Software Foundation Java projects was documented, and the feature-engineering, model-development, and framework-construction processes were described in full. The evaluation approach was set out across quantitative model evaluation, ablation studies, comparative benchmarking, and a usability assessment, and the ethical considerations that governed all data handling and participant engagement were stated.

References

- [Ajeigbe and Emma, 2021] Ajeigbe, K. and Emma, O., Visualizing Code Complexity and Maintainability with AI, 2021.
- [Al Haque et al., 2025] Al Haque, E., Brown, C., LaToza, T.D. and Johnson, B., Towards Decoding Developer Cognition in the Age of AI Assistants, 2025.
- [Alqadi and Maletic, 2020] Alqadi, B.S. and Maletic, J.I., Integration of Program Slicing with Cognitive Complexity for Defect Prediction, 2020.
- [Auprasert and Limpiyakorn, 2011] Auprasert, B. and Limpiyakorn, Y., Structuring Cognitive Information for Software Complexity Measurement, 2011.
- [Azeem et al., 2024] Azeem, M.I., Shafiq, S., Mashkoor, A. and Egyed, A., Code smells in pull requests: An exploratory study, 2024.
- [Banyongrakkul and Phoomvuthisarn, 2023] Banyongrakkul, P. and Phoomvuthisarn, S., Multi-Output Learning for Predicting Evaluation and Reopening of GitHub Pull Requests on Open-Source Projects, 2023.
- [Berciu and Moldovan, 2023] Berciu, L. and Moldovan, V., Software Maintainability and Refactorings Prediction Based on Technical Debt Issues, 2023.
- [Calixto et al., 2025] Calixto, F.E., Araújo, E.C. and Alves, E.L.G., Technical Debt in Pull Requests: Insights from Apache Projects, 2025.
- [Campbell, 2018] Campbell, G.A., Cognitive Complexity: An Overview and Evaluation, SonarSource, 2018.

[Crasso et al., 2016]	Crasso, M., Mateos, C., Zunino, A., Misra, S. and Polvorín, P., Assessing Cognitive Complexity in Java-Based Object-Oriented Systems: Metrics and Tool Support, 2016.
[Dantas et al., 2023]	Dantas, C.E.C., Rocha, A.M. and Maia, M.A., How do Developers Improve Code Readability? An Empirical Study of Pull Requests, 2023.
[De Silva et al., 2024]	De Silva, D.I., Wijesundara, D.A.R., Perera, V.H.P., Rathnayake, M.R.T.N., Udumulla, C.J. and Godapitiya, V., Cognitive Complexity Analysis and Optimization Tool for Java, 2024.
[Dossas, 2025]	Dossas, S., Software Quality Evaluation: A Comparative Study of Code Quality Analysis Tools and an Assessment of AI-Generated Code, Diploma Thesis, 2025.
[Esposito et al., 2024]	Esposito, M., Jane, A., Kilamo, T. and Lenarduzzi, V., Early Career Developers' Perceptions of Code Understandability: A Study of Complexity Metrics, 2024.
[Gonzalez-Disla, 2023]	Gonzalez-Disla, R.R., Rethinking of the Software Complexity Metrics: The Approach of Cognitive Science and Deep Learning, 2023.
[Grodzicka et al., 2022]	Grodzicka, H., Ziobrowski, A., Łakomiak, Z., Kawa, M. and Madeyski, L., Code smell prediction employing machine learning meets emerging Java language constructs, 2022.
[Gupta et al., 2024]	Gupta, M., Rajnish, K. and Bhattacharjee, V., Cognitive Complexity and Graph Convolutional Approach Over Control Flow Graph for Software Defect Prediction, 2024.

[Hassan et al., 2025]	Hassan, A.M., Abdullahi, I.R. and Hussein, A., AI-driven Defect Prediction and Automated Refactoring Framework for Large Scale Software Systems, 2025.
[Jayakuru and Wijesinghe, 2023]	Jayakuru, H.A. and Wijesinghe, D., Unified Software Complexity Measurement: A Cognitive Metric and Automated Analysis Tool, 2023.
[Kapur and Sodhi, 2018]	Kapur, R. and Sodhi, B., Estimating Defectiveness of Source Code: A Predictive Model Using GitHub Content, 2018.
[Lafi et al., 2020]	Lafi, M., Al-Dasoqi, A.B., Botros, J.W., Al-Tamimi, A. and Kafaween, H., Code Smells Analysis Mechanisms, Detection Issues, and Effect on Software Maintainability, 2020.
[Lal and Pahwa, 2022]	Lal, H. and Pahwa, G., Code Review Analysis of Software System Using Machine Learning Techniques, 2022.
[Maletic and Alqadi, 2020]	Maletic, J.I. and Alqadi, B.S., The Relationship Between Cognitive Complexity and the Probability of Defects, 2020.
[Muñoz Barón et al., 2020]	Muñoz Barón, M., Wyrich, M. and Wagner, S., An Empirical Validation of Cognitive Complexity as a Measure of Source Code Understandability, 2020.
[Noor et al., 2025]	Noor, N., Malik, N.A., Zubair, M., Safi, A., Gul, L., Khan, M.H. and Batool, S.N., Intelligent Automation in Software Engineering: Transforming Developer Productivity and Code Quality Through Machine Learning, 2025.

[Odeh et al., 2024]	Odeh, A.H., Odeh, M., Odeh, H. and Odeh, N., Measuring Cyclomatic Complexity of Source Code Using Machine Learning, 2024.
[Phaithoon et al., 2024]	Phaithoon, S., Wongnil, S., Pussawong, P., Choetkiertikul, M., Ragkhitwetsagul, C., Sunetnanta, T., Maipradit, R., Hata, H. and Matsumoto, K., FixMe: A GitHub Bot for Detecting and Monitoring On-Hold Self-Admitted Technical Debt, 2024.
[Pinto et al., 2024]	Pinto, G., de Souza, C.R.B., Neto, J.B., de Souza, A., Gotto, T. and Monteiro, E., Lessons from Building StackSpot AI: A Contextualized AI Coding Assistant, 2024.
[Prokofiev and Savenko, 2023]	Prokofiev, I. and Savenko, O., Method of Statistical Analysis of Software Code Based on Complex Parameters, 2023.
[Saborido et al., 2022]	Saborido, R., Ferrer, J., Chicano, F. and Alba, E., Automatizing Software Cognitive Complexity Reduction, 2022.
[Saborido et al., 2024]	Saborido, R., Ferrer, J. and Chicano, F., Automatizing Software Cognitive Complexity Reduction through Integer Linear Programming, 2024.
[Sergeyuk et al., 2026]	Sergeyuk, A., Izadi, M., Kuo, N. and Chen, V., Developer Interaction Patterns with Proactive AI: A Five-Day Field Study, 2026.
[Shehab et al., 2015]	Shehab, M.A., Tashtoush, Y.M., Hussien, W.A., Alandoli, M.N. and Jararweh, Y., An Accumulated Cognitive Approach to Measure Software Complexity, 2015.

[Singh and Bansal, 2023]	Singh, R. and Bansal, M., Predictive Software Quality Analysis Using Targeted Metrics and Machine Learning Models, 2023.
[Thamburaj and Aloysius, 2015]	Thamburaj, T.F. and Aloysius, A., Models for Maintenance Effort Prediction with Object-Oriented Cognitive Complexity Metrics, 2015.
[Vivanco and Jin, 2015]	Vivanco, R.A. and Jin, D., Selecting Object-Oriented Source Code Metrics to Improve Predictive Models Using a Parallel Genetic Algorithm, 2015.
[Wang et al., 2024]	Wang, Y., Hu, S., Yin, L. and Zhou, X., Using Code Evolution Information to Improve the Quality of Labels in Code Smell Datasets, 2024.
[Yang et al., 2022]	Yang, Y., Lo, D., Grundy, J., Bi, T. and Yang, X., Predictive Models in Software Engineering: Challenges and Opportunities, 2022.